

Hart am Wind

Aufgrund des Klimawandels haben die Trainer der AOI beschlossen, zur nächsten IOI mit dem Boot zu reisen. Da die Trainer jedoch zu faul sind, die Reise zu planen, haben sie dich zum Navigator ernannt und dir die Aufgabe gegeben, die minimale Anzahl an Tagen zu ermitteln, die benötigt wird, um Bolivien zu erreichen. Dein Boot befindet sich aktuell an der Position (x_1, y_1) , das Ziel liegt bei (x_2, y_2) .

Zunächst schaust du dir die Wettervorhersage an — ein String s der Länge n , bestehend nur aus den Buchstaben U, D, L und R. Jeder Buchstabe steht für eine Windrichtung. Die Vorhersage ist periodisch: Am ersten Tag weht der Wind in Richtung s_1 , am zweiten Tag s_2 , am n -ten Tag s_n und am $(n+1)$ -ten Tag beginnt sie wieder bei s_1 , und so weiter.

Die Koordinaten des Schiffs verändern sich folgendermaßen:

- Weht der Wind in Richtung U, bewegt sich das Schiff von (x, y) zu $(x, y+1)$;
- Weht der Wind in Richtung D, bewegt sich das Schiff von (x, y) zu $(x, y-1)$;
- Weht der Wind in Richtung L, bewegt sich das Schiff von (x, y) zu $(x-1, y)$;
- Weht der Wind in Richtung R, bewegt sich das Schiff von (x, y) zu $(x+1, y)$.



Zusätzlich kannst du dem Kapitän jeden Tag Anweisungen geben, das Schiff in eine der vier Richtungen zu steuern oder es auf der Stelle zu lassen. Wenn das Schiff gesteuert wird, bewegt es sich genau 1 Einheit in die gewählte Richtung. Die Bewegung des Schiffs und der Einfluss des Winds addieren sich. Wenn das Schiff stillsteht, wirkt nur der Wind.

Beispiel: Weht der Wind in Richtung U und wird das Schiff in Richtung L gesteuert, bewegt es sich von (x, y) nach $(x-1, y+1)$. Wird das Schiff zusätzlich in Richtung U gesteuert, landet es bei $(x, y+2)$.

Implementierungsdetails

```
long long calculate_days(int x1, int y1, int x2, int y2, int n,
    std::string s);
```

- x_1, y_1 — die Startkoordinaten des Schiffs.
- x_2, y_2 — die Zielkoordinaten.
- n — die Länge der Wettervorhersage.

- s — eine Zeichenkette der Länge n , bestehend nur aus den Buchstaben U, D, L und R, die die Windrichtungen an jedem Tag beschreibt.
- **Rückgabewert:** Die minimale Anzahl an Tagen, um das Ziel zu erreichen, oder -1 , wenn das Ziel unerreichbar ist.

Beispielgrader

Der Beispielgrader liest die Eingabe im folgenden Format:

Die erste Zeile enthält zwei ganze Zahlen x_1, y_1 ($0 \leq x_1, y_1 \leq 10^9$) — die Startkoordinaten des Schiffs.

Die zweite Zeile enthält zwei ganze Zahlen x_2, y_2 ($0 \leq x_2, y_2 \leq 10^9$) — die Zielkoordinaten.

Die dritte Zeile enthält eine ganze Zahl n ($1 \leq n \leq 10^5$) — die Länge des Strings s .

Die vierte Zeile enthält den String s , bestehend nur aus den Buchstaben U, D, L und R.

Die Ausgabe besteht aus einer einzelnen Zeile — die minimale Anzahl an Tagen, um das Ziel zu erreichen, oder -1 , falls es unmöglich ist.

Beispiel

Eingabe	Ausgabe
0 0 4 6 3 UUU	5

Eingabe	Ausgabe
0 3 0 0 3 UDD	3

Eingabe	Ausgabe
0 0 0 1 1 L	-1

Subtasks

Allgemein gilt:

- $0 \leq x_1, y_1 \leq 10^9$
- $0 \leq x_2, y_2 \leq 10^9$
- $x_1 \neq x_2$ or $y_1 \neq y_2$
- $1 \leq n \leq 10^5$

Du erhältst 10% der Punkte, wenn du korrekt entscheidest, ob das Ziel erreichbar ist. Formal bedeutet das: Wenn die richtige Antwort -1 ist und dein Programm -1 ausgibt, oder wenn die Antwort nicht -1 ist und dein Programm eine andere Zahl als -1 ausgibt, erhältst du Teilpunkte.

Subtask 1 (17 Punkte): $n = 1$

Subtask 2 (14 Punkte): Falls das Ziel erreichbar ist, ist die minimale benötigte Anzahl an Tagen kleiner oder gleich n

Subtask 3 (69 Punkte): Keine Einschränkungen

Limits

Zeitlimit: 0.1 s

Speicherlimit: 256 MB